A Cost-Efficient Large Language Models Based Intelligent Tutor through Caching and Local Context

Rupasinghe T.T.V.N.
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e17297@eng.pdn.ac.lk

Kalpana M.W.V.
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e17148@eng.pdn.ac.lk

Manohara H.T.
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e17206@eng.pdn.ac.lk

Jayarathne M.G.S.K
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
sonalijayarathne@outlook.com

Fonseka H.F.S.R
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e21140@eng.pdn.ac.lk

Dileka L.R.C.N.
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e21096@eng.pdn.ac.lk

Roshan G. Ragel
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
roshanr@eng.pdn.ac.lk

Isuru Nawinne
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
isurunawinna@eng.pdn.ac.lk

Damayanthi Herath*
Dept. of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
damayanthiherath@eng.pdn.ac.lk

***Abstract*—Recent advances in artificial intelligence (AI), particularly in generative AI models and large language models (LLMs), have established new opportunities for enhancing educational systems. Applications such as intelligent tutoring systems, chatbots, virtual assistants, and content generation platforms can automate grading and produce tailored educational content. Despite their advantages, challenges such as inappropriate outputs, privacy concerns, and high operational costs remain. To reduce the high operational cost of LLM platforms, this paper presents a novel, cost-effective intelligent tutoring architecture for a Computer Architecture course. By introducing a caching mechanism and incorporating local context from a specific course, the proposed architecture demonstrates the potential to reduce operational costs while aiming to maintain effective learning support. Furthermore, the solution demonstrates a reduction of seventy percent in external API calls for the evaluated twenty prompt workload, making it an effective and cost-efficient intelligent tutoring system.**

***Index Terms*—Large Language Models (LLM), Generative AI, Cost optimization, Intelligent Tutoring Systems**

I. Introduction

Large Language Models (LLMs) have become an important part of modern Artificial Intelligence (AI), enabling human-like text generation and natural language interaction [1]–[4]. LLMs support content generation, automated assistance, problem solving, and Intelligent Tutoring Systems (ITS), including personalized learning, grading, coding support, and academic guidance [5]–[12]. They also boost access through chatbots and virtual assistants to answer questions and clarify complex concepts, as well as offer multilingual learning [13]–[15]. Despite these benefits, the widespread implementation of LLM-based educational systems is hindered by issues including the reliability of responses, privacy considerations, maintenance expenses, and most significantly, API utilization expenses [16]–[18]. Frequent interactions with commercial LLM APIs can become expensive due to token and request-based pricing, creating financial constraints when scaling educational systems [18]–[23]. Although previous studies mainly focused on improving educational effectiveness, limited attention has been given to reducing the operational cost of LLM-based tutoring systems [23]. This paper presents a cost-aware tutoring architecture that reduces dependency on external LLM APIs. It integrates semantic caching with Least Frequently Used (LFU) eviction, course-specific semantic retrieval, a local extractive Question Answering (QA) model, and a confidence-based LLM fallback into a model-independent optimization layer, tested on a Computer Architecture tutoring prototype. Unlike previous solutions focused on learning effectiveness, the proposed architecture aims to minimize external LLM API usage. Experiments show a 70% reduction in external API calls for a 20-prompt Computer Architecture workload compared with an LLM-only baseline.

II. Literature Review

With the growing availability of fee-based LLMs (GPT-4, ChatGPT, J1-Jumbo), various pricing models have emerged [24]. However, processing large volumes of queries and text

979-8-3195-1461-5/26/$31.00 © 2026 IEEE

can be costly [20]. The three primary LLM cost-reduction methods are prompt adaptation, LLM approximation, and LLM cascading. Prompt adaptation, including prompt selection, query concatenation, and reducing redundant calculations and API calls, improves query efficiency. LLM approximation reduces costs through completion caching and model fine-tuning, enabling compact or specialized models. LLM cascading uses hierarchical models, where low-cost models handle most queries and high-cost models are invoked only when necessary. Measuring LLM API usage costs is also essential for assessing and comparing optimization strategies.

A. Prompt Adaptation

To maximize cost-efficiency and reduce inference costs from long inputs, prompt adaptation optimizes prompt size and design. One approach is prompt selection as demonstrated in Fig. 1, which excludes irrelevant examples and redundant information to shorten the prompt. Another approach is query concatenation, as shown in Fig. 2, which bundles multiple queries into a single prompt. This consolidates API calls and eliminates the redundant prompt overhead of processing queries individually.

B. LLM Approximation

LLM approximation reduces operational cost by minimizing the number of external API calls. One way is completion caching as shown in Fig. 3, in which responses returned by the LLM are stored locally; new queries are compared to the previous responses, and if there is a match, the response is returned from cache, or the API is called only if there is no match. This is a very effective method for repetitive tasks such as search [20]. An alternative method is model fine-tuning (Fig. 4), which involves gathering answers from a more costly LLM to train a smaller and more cost-effective model [20]. This optimized model subsequently answers future queries and thereby reduces the overall cost by decreasing the complexity of the prompt and computational resources [20].

C. LLM Cascade

To optimize cost-efficiency and performance, data-adaptive selection routes queries through a sequence of two or more LLMs, as shown in Fig. 5. When the response is reliable, based on a scoring function learned from the training set, a generation model and LLM router flag it as such and returns the answer to the caller, otherwise, it passes the query to the next model; this can be formulated as a constrained optimization problem [20]. Furthermore, using cost-efficient query-specific models to fine-tune prompts can provide further efficiency improvements. Using several optimization techniques, on the other hand, leads to an increase in training cost and to different trade-offs between the query cost, task performance, and computational resource utilization [20].

D. Cost Measurement when accessing LLM APIs

To measure the proposed system's cost impact, a historical baseline was established using 12 LLM APIs available in

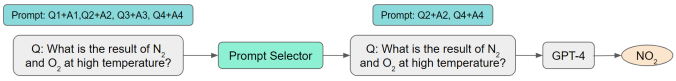


Fig. 1: Prompt Selection

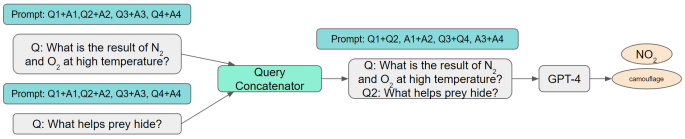


Fig. 2: Query Concatenation

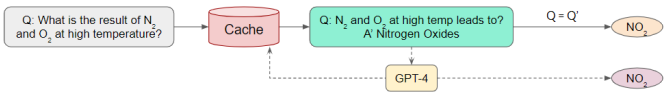


Fig. 3: Completion Cache

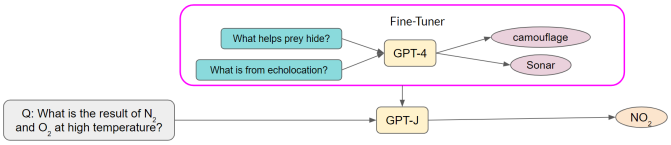


Fig. 4: Model fine-tuning

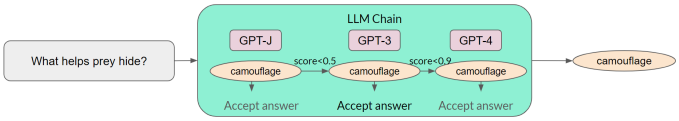


Fig. 5: LLM Cascade

March 2023 (Table I), with OpenAI pricing taken from its official pricing page [25]. Prices ranged from Textsynth GPT-J ($0.02 per 1M tokens) to OpenAI GPT-4 ($30.00 per 1M tokens), highlighting early LLM market volatility. To assess the architecture's continued relevance, frontier and lightweight models currently available, with reported prices from OpenAI and Anthropic [25], [26], are presented in Table II. The current market distinguishes high-performance reasoning models from low-cost, efficient models. For the evaluated workload, the proposed architecture reduced API calls by 70% through semantic caching at the orchestration layer.

E. FrugalGPT reduces cost and improves accuracy

FrugalGPT is a practical LLM cascade system with low inference cost and comparable or superior accuracy. It costs up to 98% less than GPT-4 [20]. It trains and evaluates on 12 LLM APIs and different datasets, as presented in Table III and Fig. 6, on low-cost models like GPT-J and T5-L and only routes queries to GPT-4 when necessary. As presented in

TABLE I: Summary of commercial LLM APIs (2023)

API	Size/B	10M input tokens (USD)	10M output tokens (USD)	request (USD)
GPT-Curie	6.7	2	2	0
ChatGPT	NA	2	2	0
GPT-3	175	20	20	0
GPT-4	NA	30	60	0
J1-Large	7.5	0	30	0.0003
J1-Grande	17	0	80	0.0008
J1-Jumbo	178	0	250	0.005
Xlarge	52	10	10	0
QA	16	5.8	5.8	0
GPT-J	6	0.2	5	0
FAIRSEQ	13	0.6	15	0
GPT-Neox	20	1.4	35	0

TABLE II: Summary of commercial LLM APIs (2026)

API	Size/B	10M input tokens (USD)	10M output tokens (USD)	request (USD)
gpt-5.6-sol	NA	50	300	0
gpt-5.6-terra	NA	25	150	0
gpt-5.6-luna	NA	10	60	0
gpt-5.5	NA	50	300	0
gpt-5.5-pro	NA	300	1800	0
gpt-5.4	NA	25	150	0
gpt-5.4-mini	NA	7.5	45	0
gpt-5.4-nano	NA	2	12.5	0
Claude Fable 5	NA	100	500	0
Claude Mythos 5	NA	100	500	0
Claude Opus 4.8	NA	50	250	0
Claude Opus 4.7	NA	50	250	0

TABLE III: Summary of Datasets Used in the FrugalGPT LLM Cascade Experiments

Dataset	Domain	Size	No. of Examples in the prompt
HEADLINES	Finance	10000	8
OVERRULING	Law	2400	5
COQA	Passage Reading	7982	2

Table IV, this method saves approximately 80% on costs and makes an accuracy gain of 15% over the use of GPT-4 alone, which is useful for optimizing LLM API usage.

F. Techniques of Checking Similarity

1) Cosine Similarity: Cosine similarity is a measure of similarity between two vectors based on their direction, rather than their magnitude [27], [28]. It measures the cosine of an angle θ between vectors A and B, providing a normalized value of the angle between the two vectors without considering the length of the vectors [29]. In mathematics, it is defined as the dot product of the vectors divided by the product of their Euclidean norms:

$$\text{Similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} \tag{1}$$

The resulting values range from -1 to $+1$, where:

TABLE IV: Overall performance and cost

Approach	Accuracy	Cost($)
GPT-4	0.857	33.1
FrugalGPT	0.872	6.5

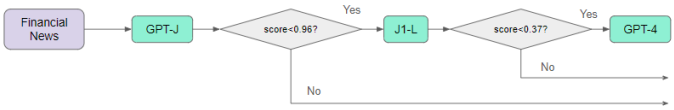


Fig. 6: FrugalGPT Strategy

- 1 indicates identical orientation (0° angle).
- 0 indicates orthogonality or no correlation (90° angle).
- -1 indicates diametric opposition (180° angle).

2) Jaccard Similarity: Jaccard similarity measures how similar two sets are [30], [31]. Used for comparing binary vectors or sets of items, the index J ranges from 0 to 1, where a value closer to 1 signifies more similarity between the two sets. Commonly used in Data Science and Machine Learning applications like Text Mining and Recommendation Systems, it is calculated as the ratio of the intersection of sets A and B to their union, as shown in (2).

$$\text{Jaccard Similarity}(A, B) = J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{2}$$

If two datasets share the same elements, their Jaccard Similarity is 1; if there are no common elements, the index is 0. Essentially, Jaccard Similarity provides insights into how much features overlap between datasets.

G. Cache Eviction Policies

Cache eviction policies are algorithms employed to manage the data stored in a cache, a high-speed temporary storage layer that enhances system performance by storing frequently and recently accessed data. These policies dictate what should be removed from the cache when it becomes full in order to free up space for new data. This process maintains an efficient replacement strategy for information in the cache, thus allowing continuous cache operation.

1) Least Recently Used (LRU): The LRU policy evicts the item that has remained unused for the longest period, assuming recently accessed items will be needed again soon. It is implemented using a linked list. Items are moved to the head upon access, and eviction occurs at the tail [32].

2) LFU: The LFU eviction policy removes the least frequently used cache item by maintaining a frequency counter for each item. The cache can be managed using a priority queue or a hash map with doubly linked lists to track frequencies and remove the lowest-frequency item during eviction [32]. Compared to LRU, LFU reduces cache pollution by retaining frequently accessed data and performs well with cyclic access patterns. However, maintaining frequency information introduces computational overhead, and LFU may suffer from

aging, where old items with high counts remain cached even when workload changes [33].

3) First in, First out (FIFO): The FIFO policy evicts the oldest item based strictly on insertion order. Implemented via a queue, new data enters at the end, and evictions occur at the front when the cache is full [34].

4) Time To Live (TTL): The TTL policy evicts items after a set period, assuming newer information is more useful than expired data. Each item receives an expiration timestamp based on its insertion time and TTL value. Managed via a priority queue or sorted doubly linked list, the expired items are evicted first to maintain up-to-date data [35].

III. Methodology

The proposed system adopts a multi-layered architecture to process student queries while minimizing reliance on external LLM APIs. As shown in Fig. 7 and Fig. 8, each incoming query is initially validated through a semantic cache. If a matching or semantically similar query is found, the cached response is returned immediately. Otherwise, the query is sent to the local retrieval and QA modules. The query is forwarded to an external LLM only when the local QA model cannot generate a response above the prototype confidence threshold. A new question-response pair is added to the cache for future use. The architecture consists of an Angular frontend, a Flask backend, a semantic caching module, a similarity computation module, and a local QA model. The semantic cache stores up to four question-response entries per cache instance. This capacity is an entry limit rather than a byte limit. Each entry stores the question, response, access count, and question embedding. A user submits a query through the Angular interface. The Flask backend encodes the query using all-MiniLM-L6-v2 and compares its embedding with cached question embeddings using cosine similarity. A cache hit occurs when the highest cosine similarity is strictly greater than 0.75. The response associated with the highest similarity is returned, and the selected entry's access count is incremented by one. Newly inserted entries receive an access count of zero. The LFU replacement policy selects the entry with the lowest access count; ties are resolved by selecting the first matching row. No time-based expiration or frequency aging is implemented. If no suitable cached response is found, the system retrieves the most relevant course-material passage and supplies it, together with the query, to the local extractive QA model, bert-base-uncased-squad2. In the prototype, a threshold of 0.5 is used to decide whether to accept the local QA response or forward it to the external LLM API. This is an example setting value and not an experimental optimized or calibrated threshold value. If the response score is greater than or equal to 0.5 it is returned and cached, if less, it is passed to the external LLM API. Flask orchestrates the semantic caching, local QA, and LLM fallback, while Angular provides the student interface. Finally, the architecture is designed to support reliable responses within the evaluated 20-prompt computer architecture workload.

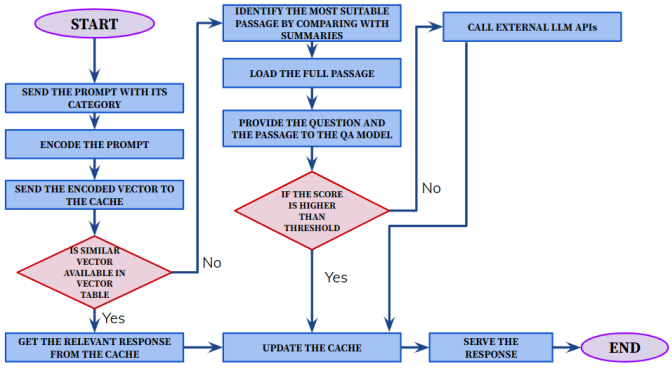


Fig. 7: Data flow diagram.

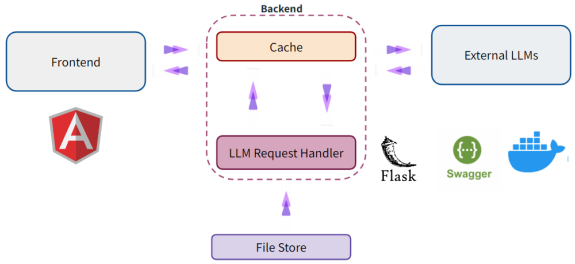


Fig. 8: High Level-Solution Architecture diagram.

IV. Experiments

The experiments used Computer Architecture course materials to evaluate the proposed intelligent tutoring architecture. The evaluation was designed as a single-course case study, with a workload of 20 prompts representing a single student interaction sequence; it was not intended to represent an institutional-scale or multi-student deployment. Two distinct evaluations were performed: an isolated evaluation of the local QA models, and an evaluation of the complete tutoring architecture, including semantic caching, LFU-based replacement, local QA, and external LLM fallback. For the isolated local QA evaluation, BERT, Electra-Base, RoBERTa, DistilBERT, DistilRoBERTa, Electra-Small, and XLNet were tested. BERT-base-uncased-squad2 achieved the highest exact-match accuracy of approximately 0.268. A confidence threshold of 0.5 was used for routing local QA responses; however, no threshold-sweep or confidence-calibration was performed, so this value of 0.5 cannot be regarded as an experimentally determined value. Queries below the threshold were forwarded to an external LLM API [36]. For the complete-system evaluation, the course materials were manually organized into a course-specific dataset containing summaries and a corresponding passages. All-MiniLM-L6-v2 was used to encode the questions and summaries. Cosine similarity was used to identify the most relevant summary for the local QA model, bert-base-uncased-squad2. The semantic cache stored 4 question-answer pairs and returned a cached response if the cosine similarity exceeded 0.75. Cache hits increased the access count by 1, while newly inserted entries started at 0. The LFU policy evicted the least-used entry; in case of a tie, the first matching entry was

chosen. All four empty entries were initialized to 0 when the backend started. However, the initial cache and reset procedure for the reported 20-prompt experiment were not recorded separately. In computer architecture, the caching mechanism reduced redundant computation and enabled response reuse for repeated student queries. These experiments were not end-to-end tests of the complete tutoring system's answer quality; they separately evaluated the local QA model and caching behavior.

A. Steps of Demonstration

Fig. 7 illustrates the execution flow. For each query, the system first searches the semantic cache. If no suitable cached response is found, the query is processed by the local QA model before being forwarded to the external LLM when necessary. Newly generated responses are stored in the cache, and the LFU policy manages cache replacement.

V. Results and Discussion

This section presents a cost analysis of the proposed intelligent tutoring system, demonstrating a 70% reduction in computational costs compared to the baseline system. The current evaluation is based on the cost reduction aspect, and accuracy analysis is recognized as an important direction for future research. The observed cost reduction demonstrates that integrating semantic caching with the QA model effectively reduces dependency on external LLM APIs while eliminating redundant query processing. The system utilizes GPT-3.5 Turbo due to its high performance, cost-effectiveness, and 16k context window optimized for conversational use [25] [37]. The cost analysis depicted in Fig. 9 is based on the GPT-3.5 Turbo pricing model (approximately 1000 tokens per 750 words). GPT-3.5 Turbo costs \$0.0010 per 1K input tokens and \$0.0020 per 1K output tokens, compared to \$0.0015 per 1K input tokens and \$0.0020 per 1K output tokens for GPT-3.5 Turbo Instruct. The evaluation workload was used to estimate the operational cost of the proposed architecture using the GPT-3.5 Turbo pricing model. During the cost analysis, the evaluation focused on measuring the number of external LLM API calls, as these directly determine the operational cost. The proposed prototype internally performs cache retrieval and local QA access before deciding whether an external API call is required. Therefore, only the number of external API calls was recorded, rather than separately reporting cache hits and cache misses. Under the proposed architecture, processing 20 prompts required only 6 external API calls instead of 20, resulting in a total cost of \$0.018 and demonstrating the effectiveness of the proposed mechanism in minimizing costly API requests. End-to-end answer quality, large-workload evaluation, detailed cache/API statistics, full token accounting, and date-verified pricing were not evaluated and remain future work.

1) Caching Mechanism and LFU Policy: The prototype employs LFU replacement to retain cached question-response pairs with higher access counts. LFU retains frequently accessed question-response pairs, while semantic similarity determines cache reuse and access frequency determines replacement within the four-entry cache. For the evaluated 20-prompt workload, the complete architecture required six external API calls, with the remaining 14 prompts handled through the cache or local QA model. This result demonstrates reduced reliance on external API calls relative to the assumed one-call-per-prompt baseline. However, because cache-served and local-QA responses were not recorded separately, the result does not quantify the independent contribution of caching or establish an advantage of LFU over alternative replacement policies.

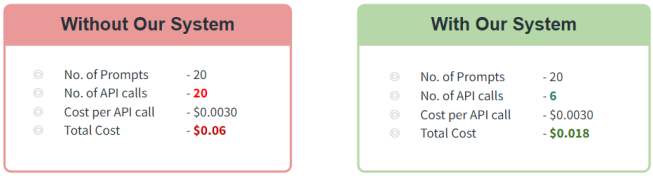
2) Confidence Threshold and Limitations: The system uses the BERT-base-uncased-squad2 model with a confidence threshold of 0.5 as a prototype routing setting. A higher threshold would lead to more low-confidence local QA responses being sent to the external LLM API, which would reduce the cost savings and also would mean that students would have fewer low-confidence local answers returned to them. In the present study, we did not perform a threshold sweep or a confidence-calibration analysis; thus, the threshold value of 0.5 should not be interpreted as optimal or calibrated.

3) Latency Considerations: Semantic caching and local QA may reduce response latency by allowing some queries to be answered without an external API request. However, the experiment did not measure latency separately for cache hits, local QA responses following cache misses, or external LLM fallback responses. The latency of each path depends on its cumulative processing overhead.

4) Limitations of the Cache Evaluation: Separate cache-hit, cache-miss, and eviction counts, together with latency measurements for individual processing paths, were not recorded for the reported experiment. Comparisons with alternative replacement policies and sensitivity analyses across cache capacities and semantic-similarity thresholds remain necessary to establish the benefit of the selected cache configuration.

$$\begin{aligned}\text{Cost} &= \text{API Calls} \times \text{Cost/Call} \\ &= 6 \times \$0.0030 = \$0.018\end{aligned}$$

5) Pedagogical Justification: The system incorporates the knowledge base of the domain into a local file store and course syllabus and into a domain-specific architecture. It addresses some of the semantic mismatches found in general-purpose LLMs by drawing answer spans from core teaching resources using an extractive model.



$$\textit{Cost Reduction} = \frac{\$0.06 - \$0.018}{\$0.06} \times 100 = 70\,\%$$

Fig. 9: Cost Analysis

While the external API calls were capped at 6 per unit of time, they cost only $0.0030 each, reducing costs by 70% of external API calls. Future research will address current limitations in domains where symbolic reasoning and visual data are required (e.g., mathematics or a circuit design) by investigating multimodal model integration and policy scalability over a large number of parallel course modules using the LFU policy.

VI. Conclusion

This paper addresses the significant operational expenses of integrating LLM into educational platforms by proposing a dynamic and context-aware caching mechanism. Experimental results demonstrate that the proposed architecture reduces dependence on external LLM APIs by reusing responses to repeated student queries through semantic caching. The demonstration was run using course material from computer architecture courses, but preliminary testing with introductory programming course material suggests possible applicability to other technical subjects. In this case study, the system successfully dealt with syntax-related queries and retrieved appropriate code snippets from the local knowledge base, but this was not a controlled evaluation and requires future validation. The outcomes of these tests point to the fact that the proposed approach could serve as a scalable and affordable framework for educational platforms powered by LLMs.

Acknowledgment

The authors thank Dr. Shamane Siriwardhana for valuable advice and feedback. ChatGPT and Gemini were used solely for language editing and stylistic refinement. All research, methodology, experiments, results, interpretations, and technical contributions were conducted and validated by the authors, who take full responsibility for the scientific claims.

References

[1] M. Wooldridge, "Intelligent agents," in *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*, 1999, vol. 1, pp. 27–73.

[2] D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press, 2010.

[3] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[4] G. Prato, E. Charlaix, and M. Rezagholizadeh, "Fully quantized transformer for improved translation," 2019.

[5] Y. Chang *et al.*, "A survey on evaluation of large language models," *arXiv preprint arXiv:2307.03109*, 2023.

[6] T. H. Kung *et al.*, "Performance of chatgpt on usmle: Potential for ai-assisted medical education using large language models," *PLoS Digital Health*, vol. 2, no. 2, p. e0000198, 2023.

[7] A. Bozkurt *et al.*, "Speculative futures on chatgpt and generative artificial intelligence (ai): A collective reflection from the educational landscape," *Asian Journal of Distance Education*, vol. 18, no. 1, 2023.

[8] X. Q. Dao, "Performance comparison of large language models on vnhsge english dataset: Openai chatgpt, microsoft bing chat, and google bert," *arXiv preprint arXiv:2307.02288*, 2023.

[9] J. G. Meyer *et al.*, "Chatgpt and large language models in academia: Opportunities and challenges," *BioData Mining*, vol. 16, no. 1, p. 20, 2023.

[10] A. Caines *et al.*, "On the application of large language models for language teaching and assessment technology," *arXiv preprint arXiv:2307.08393*, 2023.

[11] Y. Liu *et al.*, "Summary of chatgpt-related research and perspective towards the future of large language models," *Meta-Radiology*, p. 100017, 2023.

[12] V. Fernoagă *et al.*, "Intelligent education assistant powered by chatbots," *eLearning & Software for Education*, vol. 2, 2018.

[13] A. Przegalińska, "Collaborative artificial intelligence: The example of virtual assistants and conversational ai," *Artificial Intelligence (AI) as a Megatrend Shaping Education*, p. 12, 2022.

[14] P. Sridhar *et al.*, "Harnessing llms in curricular design: Using gpt-4 to support authoring of learning objectives," *arXiv preprint arXiv:2306.17459*, 2023.

[15] D. Baidoo-Anu and L. O. Ansah, "Education in the era of generative artificial intelligence (ai): Understanding the potential benefits of chatgpt in promoting teaching and learning," *SSRN 4337484*, 2023.

[16] Y. Zhou *et al.*, "Large language models are human-level prompt engineers," *arXiv preprint arXiv:2211.01910*, 2022.

[17] K. Greshake *et al.*, "More than you've asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models," *arXiv preprint arXiv:2302.12173*, 2023.

[18] E. Kasneci *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.

[19] S. S. Gill *et al.*, "Transformative effects of chatgpt on modern education: Emerging era of ai chatbots," *Internet of Things and Cyber-Physical Systems*, vol. 4, pp. 19–23, 2024.

[20] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[21] M. A. K. Raiaan *et al.*, "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," 2023.

[22] H. Yu and Y. Guo, "Generative artificial intelligence empowers educational reform: Current status, issues, and prospects," *Frontiers in Education*, vol. 8, p. 1183162, June 2023.

[23] C. Cao, "Scaffolding cs1 courses with a large language model-powered intelligent tutoring system," in *Proceedings of the International Conference on Intelligent User Interfaces (IUI)*, Mar. 2023, pp. 229–232.

[24] D. Narayanan *et al.*, "Cheaply evaluating inference efficiency metrics for autoregressive transformer apis," *arXiv preprint arXiv:2305.02440*, 2023.

[25] OpenAI, "OpenAI pricing." [Online]. Available: https://openai.com/pricing

[26] Anthropic, "Claude pricing." [Online]. Available: https://www.anthropic.com/pricing

[27] P. Xia, L. Zhang, and F. Li, "Learning similarity with cosine similarity ensemble," *Information sciences*, vol. 307, pp. 39–52, 2015.

[28] M. J. Kusner, Y. Sun, N. I. Kolkin, and K. Q. Weinberger, "From word embeddings to document distances."

[29] C. Aguerrebere, I. Bhati, M. Hildebrand, M. Tepper, and T. Willke, "Similarity search in the blink of an eye with compressed indices," Apr. 2023.

[30] G. I. Ivchenko and S. A. Honov, "On the jaccard similarity test," *Journal of Mathematical Sciences*, vol. 88, pp. 789–794, 1998.

[31] S. Niwattanakul, J. Singthongchai, E. Naenudorn, and S. Wanapu, "Using of jaccard coefficient for keywords similarity," in *Proceedings of the International MultiConference of Engineers and Computer Scientists*, vol. 1, no. 6, March 2013, pp. 380–384.

[32] D. Lee *et al.*, "On the existence of a spectrum of policies that subsumes the least recently used (lru) and least frequently used (lfu) policies," in *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, May 1999.

[33] P. R. Jelenković and A. Radovanović, "Least-recently-used caching with dependent requests," *Theoretical Computer Science*, vol. 326, no. 1–3, pp. 293–327, 2004.

[34] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and R. Vinayak, "Fifo queues are all you need for cache eviction," in *Proceedings of the 29th Symposium on Operating Systems Principles*, October 2023, pp. 130–149.

[35] N. C. Fofack, P. Nain, G. Neglia, and D. Towsley, "Performance evaluation of hierarchical ttl-based cache networks," *Computer Networks*, vol. 65, pp. 212–231, 2014.

[36] M. M. Hoque, M. A. Kabir, M. S. Rana, and T. Islam, "A multi-strategy bert-based approach for bengali question answering," in *2020 2nd International Conference on Sustainable Technologies for Industry 4.0 (STI)*, Dec. 2020, pp. 1–6.

[37] L. Spangher *et al.*, "Officelearn: An openai gym environment for reinforcement learning on occupant-level building's energy demand response," in *Tackling Climate Change with Artificial Intelligence Workshop at NeurIPS*, 2020.